

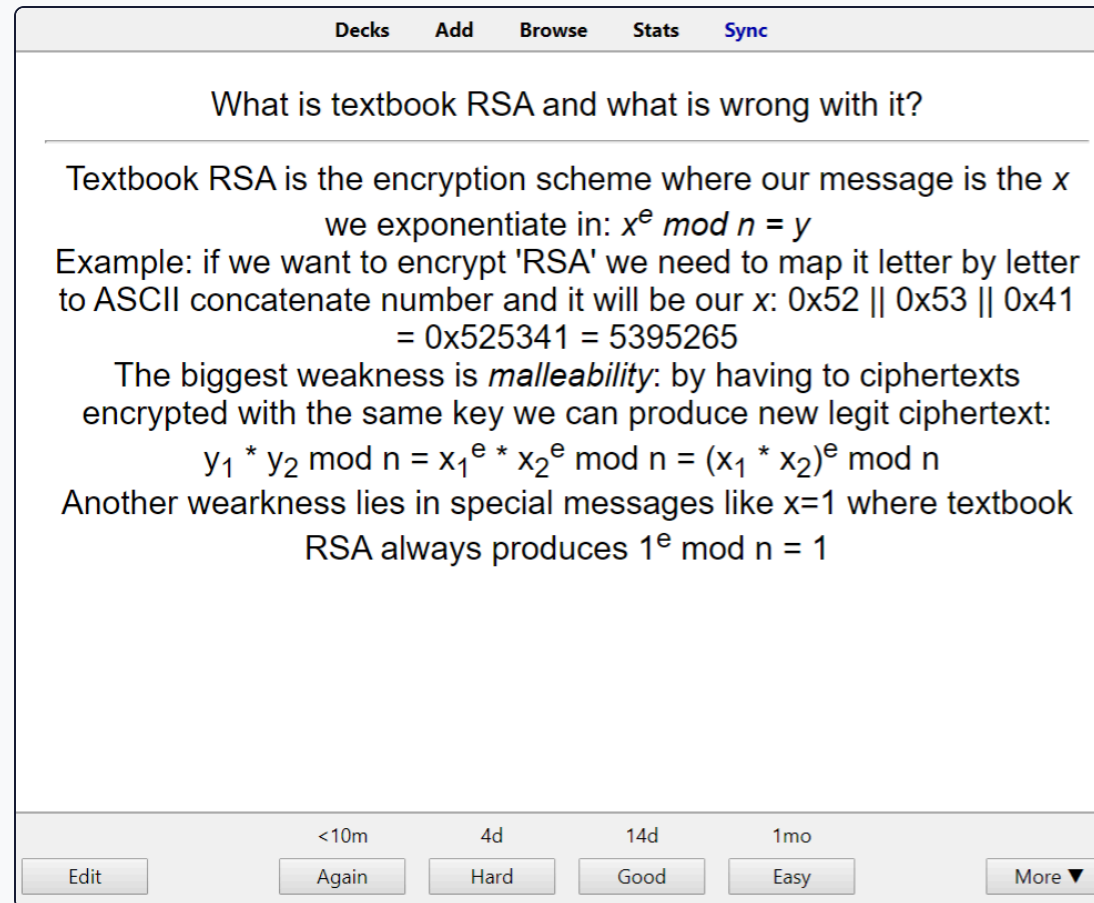
Better Flashcards

Online learning platform based on flashcards method and spiced with spaced repetition

HEIG-VD WEB 2026

Makovskyi Maksym

Anki as an inspiration source (for features)



The screenshot shows an Anki flashcard window. At the top, there is a navigation bar with tabs: Decks, Add, Browse, Stats, and Sync. The main content area displays a flashcard with the following text:

What is textbook RSA and what is wrong with it?

Textbook RSA is the encryption scheme where our message is the x we exponentiate in: $x^e \bmod n = y$

Example: if we want to encrypt 'RSA' we need to map it letter by letter to ASCII concatenate number and it will be our x : $0x52 \parallel 0x53 \parallel 0x41$
 $= 0x525341 = 5395265$

The biggest weakness is *malleability*: by having to ciphertexts encrypted with the same key we can produce new legit ciphertext:

$$y_1 * y_2 \bmod n = x_1^e * x_2^e \bmod n = (x_1 * x_2)^e \bmod n$$

Another weakness lies in special messages like $x=1$ where textbook RSA always produces $1^e \bmod n = 1$

At the bottom of the flashcard, there is a navigation bar with buttons: Edit, Again, Hard, Good, Easy, and More ▼. Above the buttons, there are time intervals: <10m, 4d, 14d, and 1mo.

From Objectives to Features

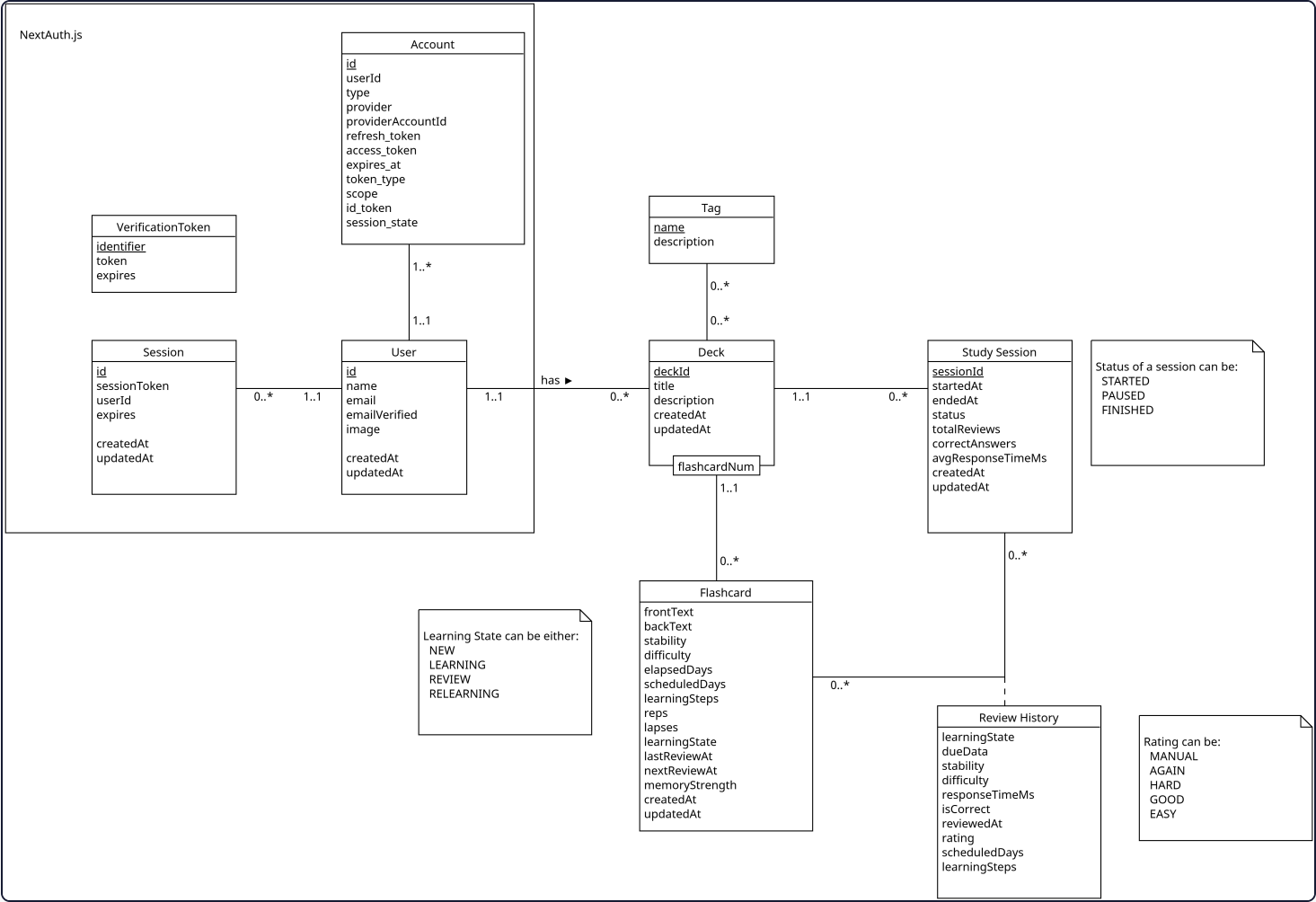
Objectives

- Create and organize personalized learning materials into structured study decks
- Improve long-term knowledge retention through adaptive repetition and review scheduling
- Track learning progress and identify weak areas over time

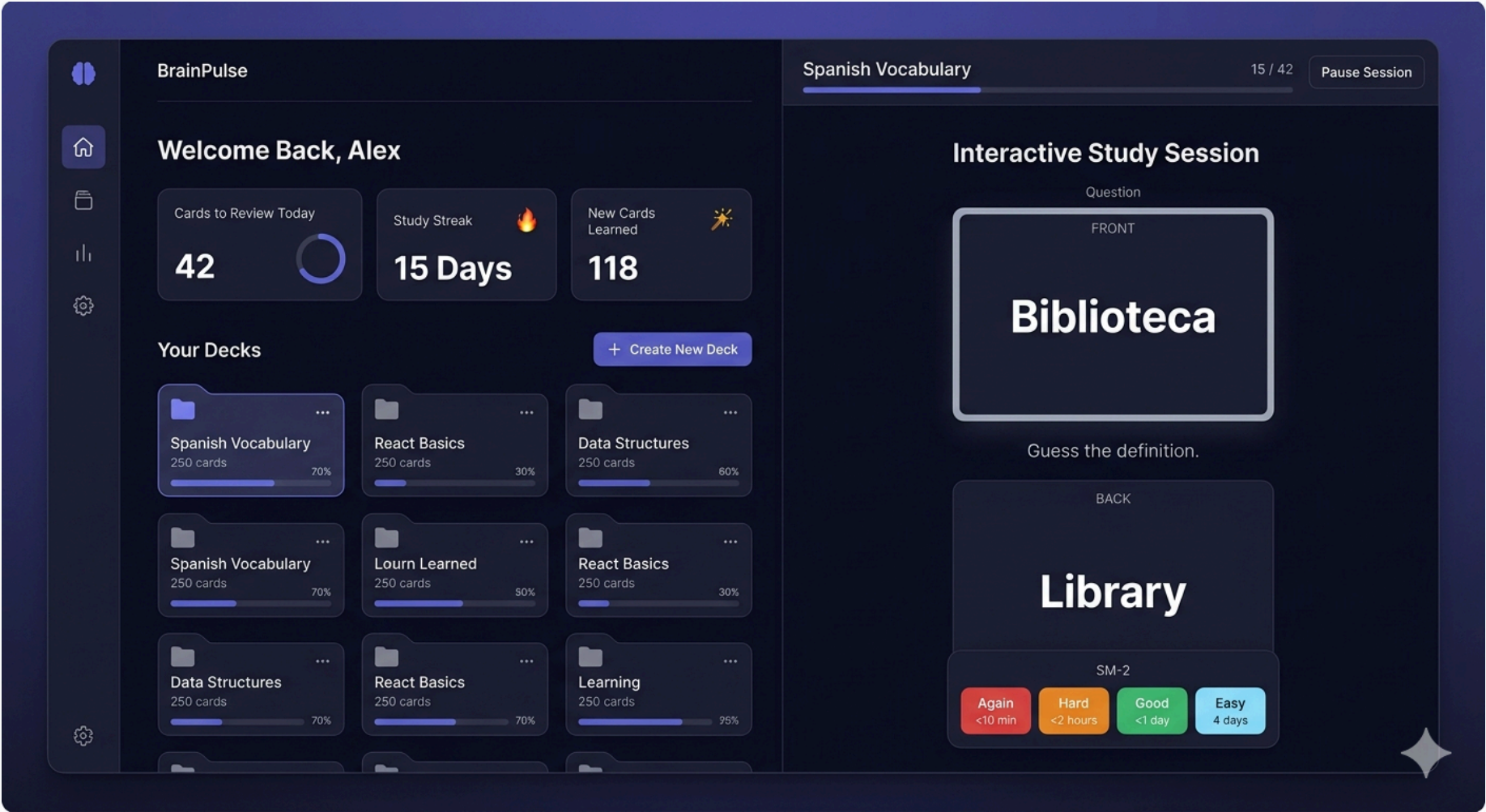
Core Features

- User authentication with identity provider (GitHub)
- Deck and Flashcard management (CRUD ops on decks/cards, cards exclusively include text)
- Adaptive study system (spaced repetition scheduling, SM-2)
- Interactive study mode (guess the other side of a card)
- Learning dashboard (track study progress, view study statistics)

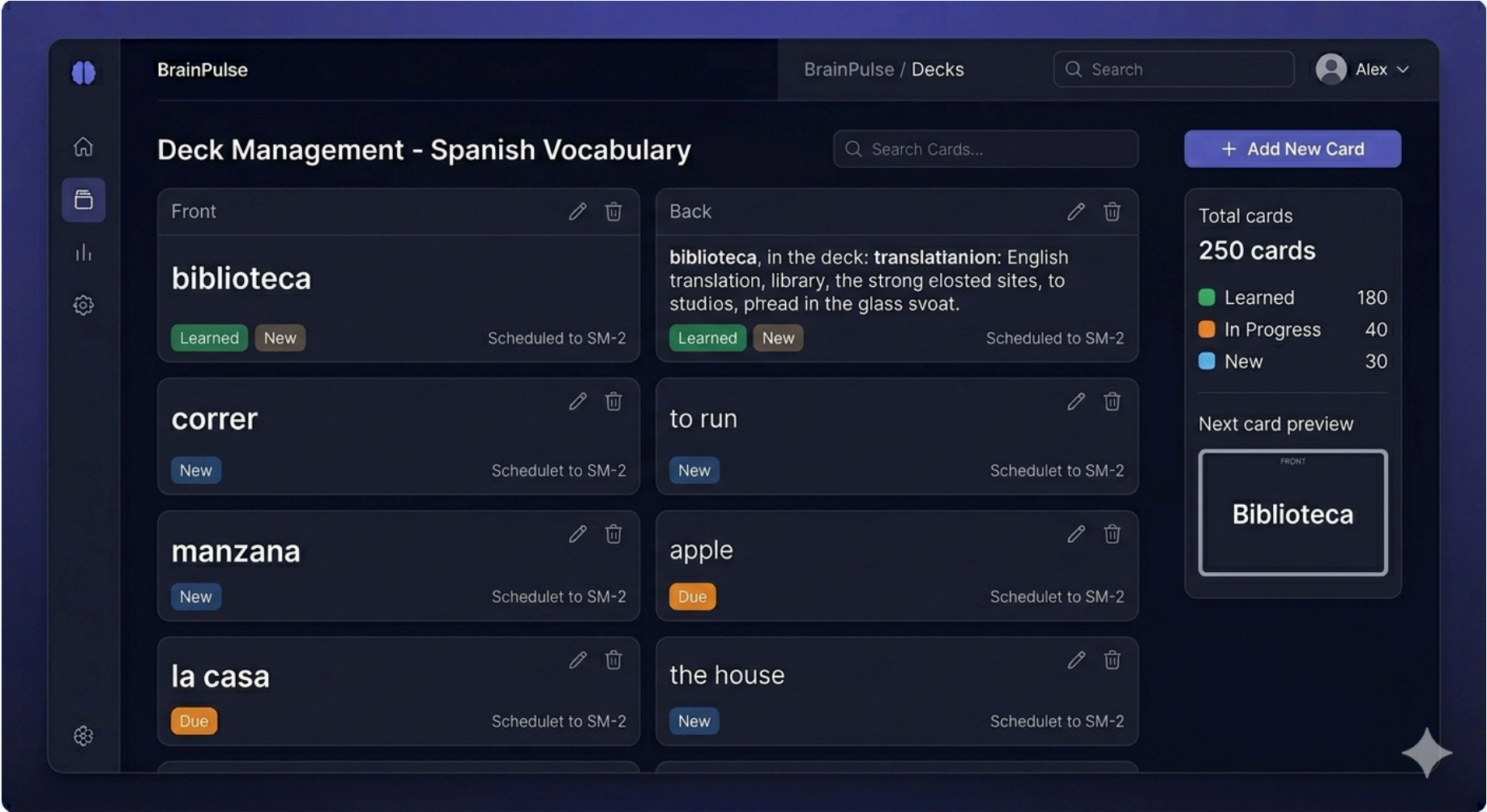
Data Model



Generated mockups (Gemini - 1)



Generated mockups (Gemini - 2)



Generated mockups (ChatGPT - 1)

BrainPulse

Dashboard

Decks

Study

Analytics

Settings

Decks > Spanish Vocabulary

Search cards...

Spanish Vocabulary

Essential Spanish words and phrases for everyday conversations.

150

Total Cards

72

Learned

42

In Progress

36

New

Created May 10, 2024

All Cards

Learned

In Progress

New

Search in deck...

All Status

All Tags

Sort by: Newest

#	Front (Spanish)	Back (English)	Status	Tags	Last Reviewed	Actions
1	hola	hello	Learned	Greetings	2 days ago	
2	gracias	thank you	Learned	Expressions	1 day ago	
3	por favor	please	In Progress	Expressions	3 days ago	
4	adiós	goodbye	In Progress	Greetings	1 week ago	
5	casa	house	New	Nouns	—	
6	comer	to eat	New	Verbs	—	
7	agua	water	New	Nouns	—	
8	amigo	friend	New	Nouns	—	

Showing 1 to 8 of 150 cards

<

1

2

3

...

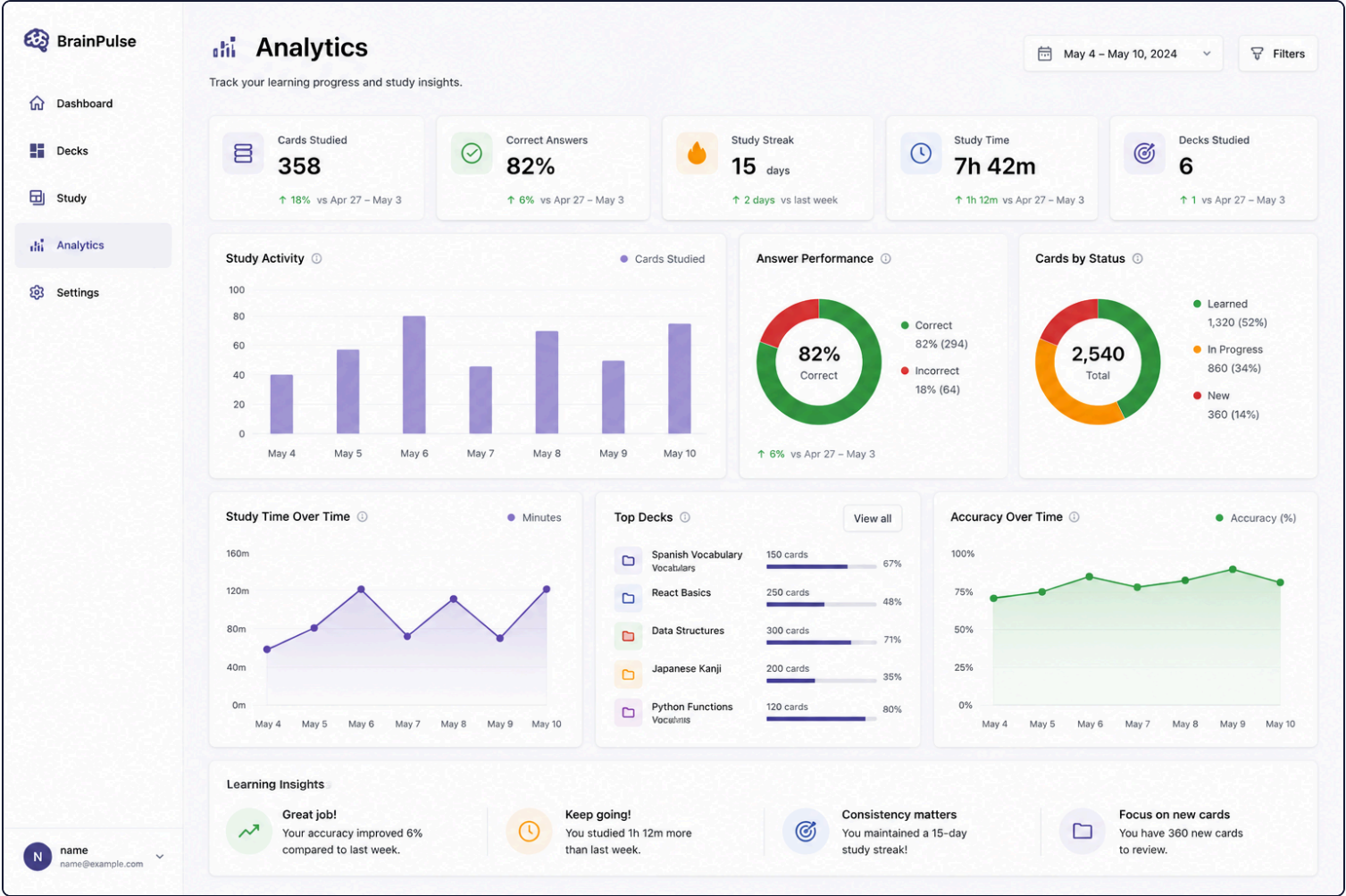
19

>

N name

name@example.com

Generated mockups (ChatGPT - 2)



Technologies selection

Initially

- Next.js
- MUI
- Zustand
- Motion React
- Prisma
- Sqlite
- NextAuth.js
- Typescript
- SuperMemo 2 (SM-2)

Currently

- Next.js
- MUI
- ~~Motion React~~
- Zustand
- Prisma
- Sqlite
- NextAuth.js
- Typescript
- ~~SM-2~~
- SWR (*new*)
- Zod (*new*)
- Free Spaced Repetition Scheduler (*new*)
- Date-fns (*new*)

Challenges (1) Types

How to type this whole thing ?

- Use Prisma types generated from models ? Is it enough ?

```
model StudySession {
  sessionId String @id @default(cuid())
  startedAt DateTime @default(now())
  endedAt DateTime?
  status StudySessionStatusEnum
  totalReviews Int @default(0)
  correctAnswers Int @default(0)
  avgResponseTimeMs Int?
  deckId String
  deck Deck @relation(fields: [deckId], references: [deckId])
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  reviewedCards ReviewHistory[]
}
```

- Not really

```
// study-store.ts:68
const session: StudySessionModel = await fetch(
  '/api/session/start',
  {
    method: 'POST',
    body: JSON.stringify({ deckId: selectedDeck?.deckId }),
  }).then(res => {
  if (!res.ok) {
    throw new Error(`Failed to create new session for deck with id=${selectedDeck?.deckId}`)
  }
  return res.json()
})
// session.startedAt is string and not a Date
// session.status is string and not enum
```

- What to do ? The answer is **Zod**

Challenges (2) Types

Zod

- Define Zod schema for `StudySession`

```
// study-session-schema.ts
export const StudySessionSchema = z.object({
  sessionId: z.string(),
  startedAt: z.coerce.date(),
  endedAt: z.coerce.date().nullable(),
  status: ???,
  totalReviews: z.number(),
  correctAnswers: z.number(),
  avgResponseTimeMs: z.number().nullable(),
  deckId: z.string(),
  createdAt: z.coerce.date(),
  updatedAt: z.coerce.date()
})
```

- What is up with `status` ?

```
// schema.prisma
enum StudySessionStatusEnum {
  STARTED
  PAUSED
  FINISHED
}
```

- Simple use `z.enum`

```
export const StudySessionStatusSchema = z.enum(StudySessionStatusEnum)
```

- Finally, use `StudySessionSchema`

```
// later in study-store.ts:73
then(res => {
  if (!res.ok) {
    throw new Error(`Failed to create new session for deck with id=${selectedDeck?.deckId}`)
  }
  return res.json()
}).then(s => StudySessionSchema.parse(s))
```

Challenges (3) Data fetching 1

- Given this endpoint, how would you fetch data ?

Endpoint	Description
<code>GET /api/decks</code>	Returns all the decks with flashcards inside

Returned type:

```
export type EnhancedDeckModel = DeckModel & { flashcards: FlashcardModel[] }
```

- Would you use `fetch`, `useEffect` and `useState` / `Context` ?
- There is something better ?

- Yes, **SWR**. It does everything above + data (re-)validation

```
// _hooks/use-all-decks.ts
const { data, error, isLoading, isValidating, mutate } =
  useSWR<EnhancedDeckModel[], Error>({
    url: '/api/decks',
    (url: string) => generalGetFetcher(url)
      .then(obj => EnhancedFlashcardsDeckArraySchema.parse(obj) as EnhancedDeckModel[])
  })
```

- But then the real challenge arises - pre-fill client component on the server
Note (from <https://react.dev/reference/rsc/use-client>):
 - Client Components are components in a render tree that are rendered on the client.
 - Server Components are components in a render tree that are rendered on the server

Challenges (3) Data fetching 2

Pre-filling data for SWR

It can be solved by fetching data in server component and passing promise as fallback data to client component, in this way entry in cache referenced by `url` used to fetch data is already prefilled when client component calls `useSWR`

- Server Component

```
export default async function DecksPage() {
  // ...
  const decksPromise: Promise<EnhancedDeckModel[]> = prisma.deck.findMany({
    where: { userId: session.user?.id },
    include: { flashcards: true }
  })
  return (
    <SWRConfig
      value={{
        fallback: {
          '/api/decks': decksPromise,
        },
      }}
    >
    <AllDecksWorkspace />
    </SWRConfig>
  )
}
```


It is demo time !

Specification

What has or has not been done according to intial design document ?

Technologies

Name	Reason
React Motion	Material design guideliness advises against Card flipping as method to reveal card content
SuperMemo 2 (SM-2)	Obsolete (1987) and not really appearing (you should wait a day for repeat card that you don't know at all)

Features

Reason is one - I've run out of time.

- Deck information editing
- (*) Filtering and Searching through collection of Decks/Cards
- (*) No way to update the algorithm settings from UI

() was not a part of initial design document but can be considered this way*

Further development

- Proper errors display
- Cards can include the images and rich text
- Tags
- Filtering and search
- Session details
- More study modes (pair it/choose it/type it)
- Public/private decks
- More charts and analytics data for dashboard (example: Knowledge Graph Visualization of related cards among related decks)
- Replay previous study session
- Shared decks

Thank you for your attention !
